

Milestone 4 - Sprint 2
CSCE3701 - Software Engineering (Fall 2025)
Group Number: 1

Members:
Rawan Muhammad
Salma El_Hawary
Doha Nour El-Din
Habeba Saad
Yara Abdelkader

1. Sprint Summary	2
2. Sprint Board Snapshot	6
3. Definition of Done (DoD) Checklist	9
4. Working Software Demo	10
5. Code Evidence	10
6. Testing	11
7. Root Cause Analysis	16
8. Final Retrospective Notes	18
9. Burndown Chart	20
10. Documents	21

1. Sprint Summary

- Sprint Goal:

The goal of this sprint was to enhance the core functionality of the system by improving user accessibility, strengthening patient interactions with appointments, and ensuring accurate data entry during the sign-up process. This sprint focused on delivering features that support account recovery, appointment management, clinic browsing, and input validation.

- Scope Overview: Key user stories or backlog items tackled.

During this sprint, the team successfully completed three user stories along with two supporting tasks:

Completed User Stories

1. Forgot Password Functionality

As a system user, I want to use a “Forgot Password” feature so that I can reset my password if I forget it and regain access to my account.

2. Reschedule Appointment

As a patient, I want to reschedule my appointment so that I can change the date or time when needed.

3. Past Appointment History

As a patient, I want to view my past appointments so that I have a history of my appointments.

4. Sign-Up and Input Validation

As a patient, I want to create a new account by providing my personal details and contact information so that I can access the hospital’s online services securely.

5. Adding Doctors' Available Slots

As a doctor, I want to set my working hours and availability so patients can book appointments during those times.

Completed Tasks (Organized by User Story)

A. Past Appointment History

- Build a Past Appointments page in the patient interface.
- Connect the UI to the controller and database to fetch past appointment data.
- Create back-end logic to filter and return only past appointments.
- Display past appointments in a clean list with date, time, and doctor details.
- Add a “No past appointments found” message when applicable.
- Implement automatic status update for appointments whose time has passed.
- Update the appointment status.
- Test cases for past appointment retrieval and display.

B. Reschedule Appointment

- Create a Reschedule button on the upcoming appointments page for each appointment.
- Implement back-end logic to update the appointment record.
- Display updated appointment details on the UI.
- Add test cases to verify the rescheduling functionality.
- Notify the patient that the appointment was successfully rescheduled.

C. Sign-Up and Input Validation

- Implement input validation rules for age, phone number, and email.
- Add checks to ensure the phone number contains only digits and follows the accepted length.
- Add front-end validation messages for incorrect or missing fields.
- Update sign-up UI to highlight invalid fields (red borders or warnings).
- Prevent submission of the sign-up form until all fields pass validation.
- Unit tests for all validation functions.

D. Forgot Password Functionality

- Add a “Forgot Password” button on the login page.
- Implement back-end logic to verify if the email exists in the system.
- Implement updating the password in the database.

Add confirmation messages for successful or failed password resets.

Expected Deliverable

The expected deliverable for this sprint was a functional increment added to the MVP, including:

1. **Forgot Password Functionality**

A working flow enabling users to recover account access.

2. **Enhanced Appointment Management**

Ability to reschedule appointments.

View past appointment history with date, time, and doctor details.

3. **Doctor Availability**

Browse doctors and their available time slots for clinics.

4. **Improved Sign-Up Validation**

Ensure correct entry of patient information through validation checks for age, phone number, email, and other personal details.

Overall, a more reliable and user-friendly version of the system aligned with MVP requirements.

2. Sprint Board Snapshot

Jira Backlog.

<https://aucegypt-team-tk4cf28k.atlassian.net/jira/software/projects/HMS/boards/1>

Sprint 2 is 100% complete. Task distribution was balanced across 5 team members

The image displays two screenshots of a Jira Sprint Board for the 'Hospital Management System' project. The board is organized into four columns: 'TO DO', 'IN PROGRESS', 'IN REVIEW', and 'DONE'. The 'DONE' column contains 5 tasks, all of which are 100% complete. The tasks are as follows:

- Task 1:** Notify the patient that the appointment was successfully rescheduled. (APPOINTMENT MANAGEMENT, HMS-155, assigned to H, 100% complete)
- Task 2:** As a doctor, I want to set my working hours and availability so patients can book appointments during those times. (DOCTOR SCHEDULE MANAGEMENT, HMS-44, assigned to S, 100% complete)
- Task 3:** Test cases for past appointment retrieval and display. (APPOINTMENT MANAGEMENT, HMS-150, assigned to DE, 100% complete)
- Task 4:** Connect the UI to the controller and database to fetch past appointment data. (APPOINTMENT MANAGEMENT, HMS-149, assigned to DE, 100% complete)
- Task 5:** Add a "No past appointments found" message when applicable. (APPOINTMENT MANAGEMENT, HMS-148, assigned to DE, 100% complete)

The bottom screenshot shows a similar sprint board with 5 tasks in the 'DONE' column, all 100% complete:

- Task 1:** Display past appointments in a clean list with date, time, and doctor details. (APPOINTMENT MANAGEMENT, HMS-147, assigned to DE, 100% complete)
- Task 2:** Build a Past Appointments page in the patient interface. (APPOINTMENT MANAGEMENT, HMS-146, assigned to DE, 100% complete)
- Task 3:** Update the appointment status. (APPOINTMENT MANAGEMENT, HMS-154, assigned to H, 100% complete)
- Task 4:** Implement automatic status update for appointments whose time has passed. (APPOINTMENT MANAGEMENT, HMS-145, assigned to DE, 100% complete)
- Task 5:** Create back-end logic to filter and return only past appointments. (APPOINTMENT MANAGEMENT, HMS-144, assigned to DE, 100% complete)

Spaces

Hospital Management System

Summary Backlog Board Code Timeline Pages Forms Releases +

Search board



Filter

TO DO

IN PROGRESS

IN REVIEW

DONE 28

Implement input validation rules for age, phone number, and email.

PATIENT MANAGEMENT

✓ HMS-138 ✓ = T

Add confirmation messages for successful or failed password resets.

STAFF ACCOUNT MANAGEMENT

✓ HMS-161 ✓ = RM

Implement updating the password in the database.

STAFF ACCOUNT MANAGEMENT

✓ HMS-160 ✓ = H

Implement back-end logic to verify if the email exists in the system.

STAFF ACCOUNT MANAGEMENT

✓ HMS-158 ✓ = RM

Add a "Forgot Password" button on the login page.

STAFF ACCOUNT MANAGEMENT

✓ HMS-157 ✓ = SE

Spaces

Hospital Management System

Summary Backlog Board Code Timeline Pages Forms Releases +

Search board



Filter

TO DO

IN PROGRESS

IN REVIEW

DONE 28

unit tests for all validation functions.

PATIENT MANAGEMENT

✓ HMS-143 ✓ = T

Update sign-up UI to highlight invalid fields (red borders or warnings).

PATIENT MANAGEMENT

✓ HMS-142 ✓ = T

Prevent submission of the sign-up form until all fields pass validation.

PATIENT MANAGEMENT

✓ HMS-141 ✓ = T

Add checks to ensure phone number contains only digits and follows accepted length.

PATIENT MANAGEMENT

✓ HMS-140 ✓ = T

Add front-end validation messages for incorrect or missing fields.

PATIENT MANAGEMENT

✓ HMS-139 ✓ = T

Implement input validation

Spaces

Hospital Management System ...

Summary Backlog **Board** Code Timeline Pages Forms Releases +

Q Search board SE DE H RM Y Filter

TO DO	IN PROGRESS	IN REVIEW	DONE 28
			time when needed. APPOINTMENT MANAGEMENT HMS-10 ✓ 5 DE
			As a system user, I want to use a "Forgot Password" feature so that I can reset my password if I forget it and regain access to my account. STAFF ACCOUNT MANAGEMENT HMS-53 ✓ 3 SE
			Create a Reschedule button on the upcoming appointments page for each appointment APPOINTMENT MANAGEMENT HMS-151 ✓ DE
			Display updated appointment details on the UI. APPOINTMENT MANAGEMENT HMS-155 ✓ DE
			Implement back-end logic to update the appointment record. APPOINTMENT MANAGEMENT HMS-152 ✓ SE

Spaces

Hospital Management System ...

Summary Backlog **Board** Code Timeline Pages Forms Releases +

Q Search board SE DE H RM Y Filter

TO DO	IN PROGRESS	IN REVIEW	DONE 28
			Add test cases to verify the rescheduling functionality. APPOINTMENT MANAGEMENT HMS-156 ✓ DE
			As a patient, I want to view my past appointments so that I have a history of my appointments APPOINTMENT MANAGEMENT HMS-135 ✓ 3 DE
			As a patient, I want to create a new account by providing my personal details and contact information so that I can access the hospital's online services securely. PATIENT MANAGEMENT HMS-137 ✓ 5 Y
			As a patient, I want to reschedule my appointment so that I can change the date or time when needed. APPOINTMENT MANAGEMENT HMS-10 ✓ 5 DE

Figures (1-6): Jira taskboard snapshot

3. Definition of Done (DoD) Checklist

The team agreed that a backlog item is considered “Done” only when it meets all of the following criteria:

Development

- The feature is fully implemented according to the user story and acceptance criteria.
- All required validations, error handling, and edge cases are covered.
- Code follows the team's coding standards and naming conventions.

Testing

- The feature has been functionally tested by the team.
- No critical or major bugs remain.

Integration

- The feature is integrated with the existing system without breaking other functionality.
- Database changes (if any) are migrated and verified.

UI/UX

- User interface matches the approved design (if designs exist).
- Input fields, buttons, and user flows work as expected.

Documentation

- Code is documented where necessary.
- The user story and task status are updated on the project board.
- Any configuration or setup steps are documented.

Review

- The feature has been peer-reviewed or approved by a team member.
- The Product Owner or team lead has verified that acceptance criteria are met.

Deployment

- The feature is merged into the appropriate branch.
- The application builds successfully without errors.
- The increment is included in the sprint deliverable or demonstration.

4. Working Software Demo

<https://youtu.be/mf28eo53y90>

5. Code Evidence

Repository link: <https://github.com/Rawan10101/Carevo>

Each team member has contributed to the development and has committed at least 1 commit.

The project utilized the **Git Feature Branching Model** to manage all source code, structuring the workflow around feature isolation. The protected main branch served as the final deployment target, while the develop branch was the central, mandatory integration point for all completed work.

All user story development was conducted on dedicated feature branches before merging into develop:

- **Authentication Branches:** Work for the **Forgot Password Functionality** and **Improved Sign-Up Validation** was implemented on specific feature branches (e.g., feature/AUTH-forgot-password).
- **Appointment Branches:** Features related to **Appointment Rescheduling** and **Viewing Past History** were developed on separate branches (e.g., feature/APPT-reschedule-logic).
- **Availability Branch:** Logic for **Doctor Availability Browsing** was done on its own dedicated feature branch.

6. Testing

- **Test Case Sheet**

https://docs.google.com/spreadsheets/d/13FPpidmYawT9PP0JMDHtEkMVSzB_Nt-ukp3pS_PCWT0/edit

- **Bug Report**

ID Number	1
Name	Patient is unable to reschedule the appointment
Reporter	Yara Abdelkader
Submit Date	12/3/2025
Summary	When I press reschedule appointment and choose a new time, its timing does not change.
Platform	iOS
Operating System	iOS 16
Browser	Expo go
Severity	Medium
Assigned to	Doha Nour El-Din
Priority	High

Description

When I press reschedule appointments and choose a new time, I find in the My appointments tab that its time did not change.

Steps to reproduce

- Log in as a patient
- Book an appointment
- Open My Appointments tab
- Press “Reschedule” for some appointment
- Choose a different timing

- Open the MY Appointments tab again

Expected Result

Seeing the appointment booked to have a different time.

Actual Result

Seeing the appointment's timing has not changed.

● **Testing Types & Techniques**

Our testing strategy employed **Black-Box Testing** techniques, which treat the application as a "black box" and examine its functionality without knowledge of the internal code structure. This approach is ideal for validating user-facing requirements and ensuring the software behaves correctly according to specifications.

We selectively applied two specific black-box techniques based on the complexity and nature of the feature being tested: Decision Table Testing for multi-conditional inputs, and Use Case Testing for end-to-end user workflows.

1. Decision Table Testing (Multi-Conditional Input Validation)

Applied to: Sign Up Form Inputs (e.g., Email, Password, Phone Number, Age, Name – **TC 01, TC 02, TC 03, TC 06, TC 09, TC 10**)

The sign-up feature involves multiple independent, yet interacting, input fields. Each field has specific, non-negotiable **conditions** (e.g., password must be long enough, email must be correctly formatted). A simple linear test or Equivalence Partitioning might miss complex interactions or edge cases.

Decision Table Testing was chosen because it is the most effective technique for:

- **Handling Complex Rules:** It systematically maps all possible input conditions to their resulting actions (outcomes). For instance, the testing included complex password rules (TC 09, TC 10), which forced the team to explicitly define outcomes for conditions like: {Length < 6} (TC 03), {Missing Letters} (TC 09), and {Missing Digits} (TC 10).
- **Ensuring Full Coverage of Logic:** By listing all valid and invalid combinations of conditions, we guaranteed that we tested not only the success path (TC 04: all fields valid) but also all relevant boundary conditions (e.g., Age minimum boundary check in TC 06) and failure paths (TC 01, TC 02).

- **Clarity and Documentation:** The resulting decision table serves as clear, formal documentation of the expected behavior for every possible input scenario, greatly simplifying bug analysis and future regression testing.

2. Use Case Testing (Workflow and Appointment Management)

Applied to: Complex User Flows and Security Sequences (e.g., Reschedule Appointment, View Past Appointments, Forget Password – **TC 07, TC 08, TC 11**)

This technique was applied to features that involve a **sequence of steps** required to achieve a real-world user goal. The success or failure of the test depends not on a single input, but on the successful execution of an entire transaction (a "use case").

Use Case Testing was selected because it allows us to test the system from the user's operational viewpoint:

- **Focus on Goals:** Instead of isolated input tests, this method validates whether the user can achieve their goals (e.g., "reschedule appointment" in TC 07 or "reset password" in TC 11) by traversing the necessary screens and system states (authentication, selection, confirmation).
- **Verifying System Integration:** Use cases inherently test the seamless integration between different parts of the application, such as the front-end UI, navigation, and the back-end database services (which retrieve past appointments for TC 08 or securely handle password tokens for TC 11).
- **Simulating Real-World Scenarios:** The test cases are derived directly from the user requirements documents. This ensures that the application is tested using the scenarios that users are most likely to encounter, maximizing the discovery of high-impact usability or flow defects.

As for overall testing of the project, we have used the following:

1. Functional Testing

We used functional testing to verify that each feature behaves exactly as required.

The reasons for choosing it:

- These sprint features are functional requirements (rescheduling, viewing history, validating inputs). Also, it ensures the software behaves as the user expects. Different levels of testing were included in functional testing.

a. Unit Testing

Developers performed unit tests on:

- Sign-up input validation functions
- Appointment rescheduling logic
- Database fetch functions for past appointments

The reasons for choosing it:

- Helps catch errors early.
- Faster to fix at this stage.
- Ensures each component works correctly before integration.

b. Integration Testing

We tested how modules interact, such as:

- Patient module ↔ Appointment module
- Appointment module ↔ Database
- Authentication ↔ View Past Appointments page

The reasons for choosing it:

- Integration issues often appear when combining modules.
- Ensures that data flows correctly across components
- Validates that updating an appointment is correctly reflected in the system database.

c. System Testing

After integrating Sprint 2 features, full system testing was performed.

The reasons for choosing it:

- Required to test the end-to-end workflow:
 - signUp → login → view appointment page → view past appointments

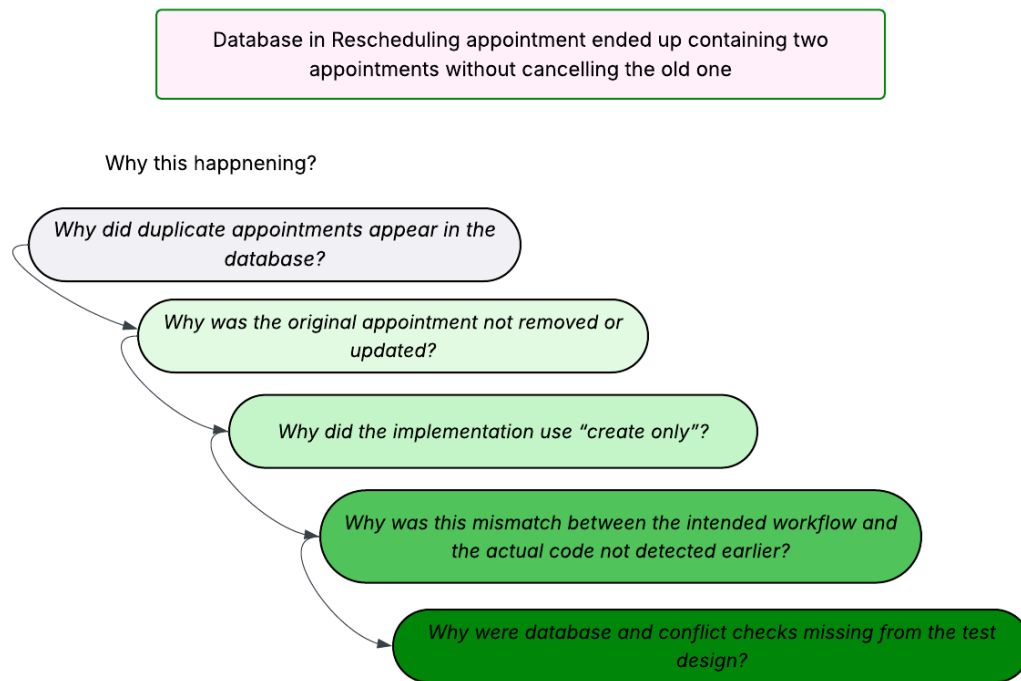
- signUp → login → view appointment page → view past appointments → reschedule → confirm.
- Checks both functional and non-functional behaviors.
- Ensures the system works correctly for real user scenarios.

7. Root Cause Analysis

During Sprint 2, we encountered a persistent defect in the appointment rescheduling feature. When a patient rescheduled an appointment, the UI showed the new date and time as expected, but **the database ended up containing two appointments: the original slot and the new one**. As a result, doctors and receptionists saw duplicate bookings in their schedules, and the old slot was never freed for other patients, violating the requirement that rescheduling should move the booking rather than create an extra one.

To understand why this kept happening, we applied the **5 Whys root cause analysis technique**. The questions were:

- First: *Why did duplicate appointments appear in the database?* Because the reschedule function added a new appointment document while leaving the original document unchanged.
- Second: *Why was the original appointment not removed or updated?* Because the implementation treated rescheduling as “create a new booking” and never called a delete or update operation for the old record.
- Third: *Why did the implementation use “create only”?* Because the developer reused the booking logic without adding a step to reference the existing appointment ID or status, so the code path was identical to normal booking.
- Fourth: *Why was this mismatch between the intended workflow (cancel old + add new) and the actual code not detected earlier?* Because early tests validated only that the UI showed the new slot and did not check for lingering records in Firestore or double-booked time slots.
- Fifth: *Why were database and conflict checks missing from the test design?* Because the initial test cases focused on user-visible behavior and did not explicitly include post-conditions about the number of appointment records or the state of the original slot.



Figure(7): Visualization to summarize the 5 WHYs

We introduced both corrective and preventive actions to solve this issue.

- **Corrective changes:** modified the reschedule logic so that it performs an atomic two-step operation:
 - First, the system updates the existing appointment document (or marks it as “Rescheduled/Cancelled”) and frees the old slot in the doctor’s schedule.
 - Second, it creates the new appointment only after the old one is safely updated, ensuring there is never more than one active appointment for the same booking.
- **Preventive measures:** included updating the test case sheet to add checks that rescheduling results in exactly one active appointment per patient–doctor pair and that the previous slot becomes available, and adding a Definition-of-Done criterion requiring verification of both UI and database state for critical workflows.

8. Final Retrospective Notes

What Went Well

We successfully delivered all the essential patient features, meeting our core goals for the project.

- **Core Feature Delivery:** We achieved successful delivery of the most critical patient features: **secure account creation, login, appointment booking, rescheduling, and cancellation**. This proves that our project goals were well-aligned and achievable.
- **Effective Technology:** Using **Firebase** for authentication and data storage proved to be a highly reliable and efficient choice, making the initial setup and data management simpler.
- **Quality Testing:** We used smart testing methods (Decision Table Testing for sign-up and Use Case Testing for appointments) to ensure that the critical features were thoroughly checked for success and error handling, resulting in a stable product.
- **Good Teamwork:** Collaboration between the front-end and back-end developers was consistently smooth, which helped translate design ideas into working screens quickly.

What Didn't Go Well

We faced challenges primarily related to time and technical depth, which slowed down our progress in the middle of the project.

- **Database Difficulty:** Our limited experience with advanced **Firestore queries** meant that we spent extra time figuring out how to retrieve and filter data efficiently (like finding available clinics and appointments). This slowdown required us to dedicate a later sprint to fixing performance issues.
- **Complex Feature Underestimation:** We severely underestimated the time needed for features that involved updating two things at once, such as the **reschedule and cancel logic**. These tasks required complex internal changes (freeing up the old slot while booking the new one) that took longer than planned.
- **Missing Error Checks:** We failed to account for all possible failure scenarios in the login and sign-up process (like trying to create a duplicate account or forgetting a password), which required extra time in later sprints to stabilize and fix.

What Should Be Improved

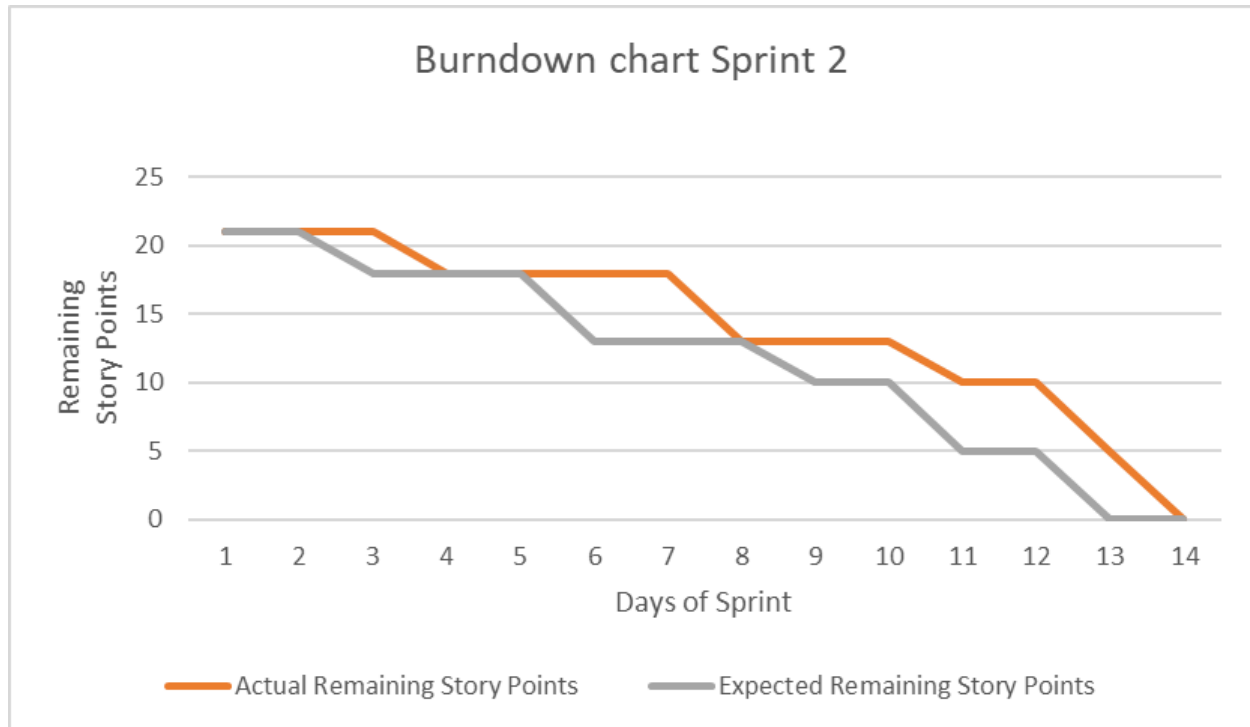
We identified clear steps to improve our efficiency and stability in future projects.

- **Deeper Tool Knowledge:** For future projects, we must invest time **before coding begins** on mandatory training to master our core technical tools, especially advanced database

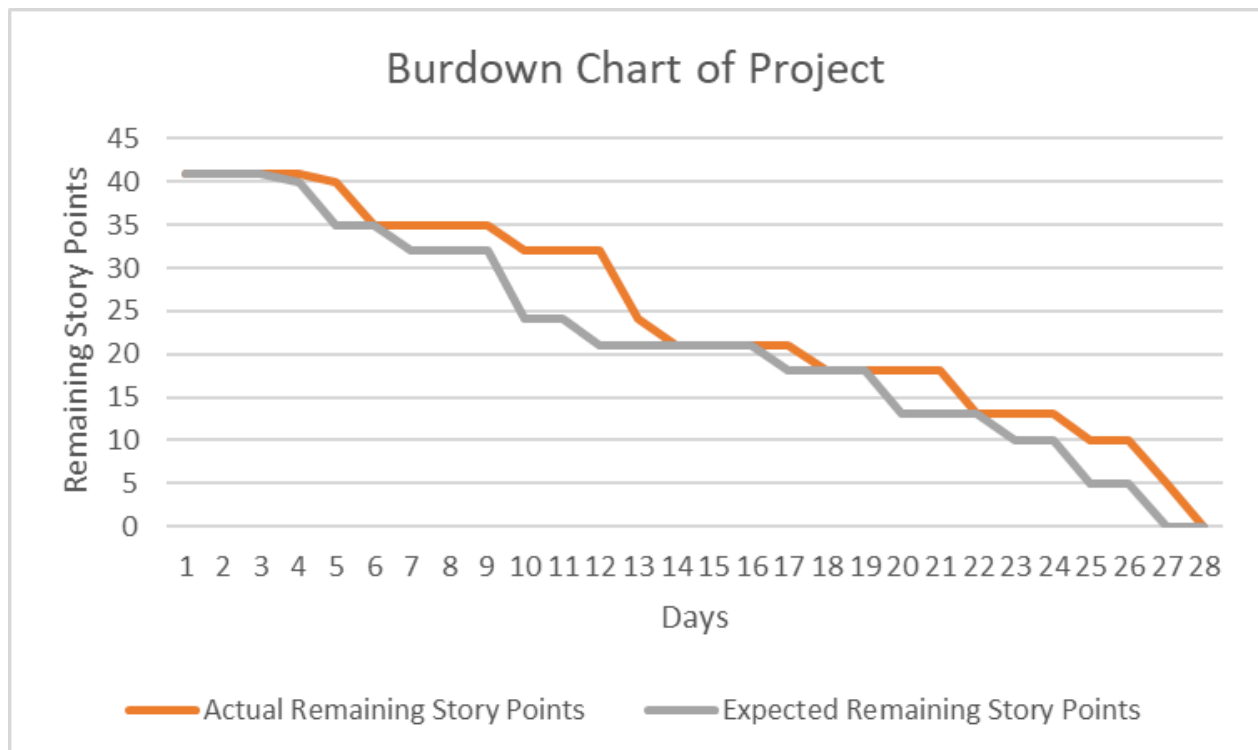
features. This preparation will prevent the slowdowns we experienced due to struggling with complex queries.

- **Automate Testing:** While our manual checks were good, ensuring quality is too slow. We need to integrate **Automated End-to-End (E2E) testing** from the start. This will automatically confirm that core flows (like signing up and booking) still work perfectly every time we change the code.
- **Better Time Estimation:** We need to be more realistic when estimating the time required for complex feature integrations. We will ensure tasks involving two-way data changes (like scheduling updates) are assigned more conservative time estimates.
- **Consistent Error Tracking:** We struggled to track down tricky bugs because our error reporting was disorganized. We must implement a comprehensive, standardized system for logging and tracking all errors from the moment a project starts. This will ensure that all failures are instantly recorded, making them much faster to fix.

9. Burndown Chart



Figure(8): Burndown chart of sprint 2



Figure(9): Burndown chart of the project

10. Documents

- Requirements specification document:

https://docs.google.com/document/d/1f1hmFtX_-Dvp_DHqrPH0mdQ6GPfLzlUduLcgVieGmNs/edit?tab=t.0#heading=h.dgz07mkfaeu2

- Design specification document:

https://docs.google.com/document/d/1_nmkvvqNb9T8_X2EBdULvE-0o9T5U5Pq0KdWdFFvKXI/edit?tab=t.0